

SPI-Based Accelerometer Interfacing Using Artix-7 FPGA

Chapter 1 - Introduction

Brief Description

This project focuses on interfacing a digital accelerometer with an Artix-7 FPGA using the Serial Peripheral Interface (SPI) protocol. The FPGA is configured as an SPI master to communicate with the accelerometer, enabling the reading of multi-axis acceleration data (X, Y, and Z) through register-based transactions. The design is implemented using Verilog HDL and follows a finite state machine (FSM)–based SPI controller architecture. The project demonstrates low-level sensor communication, synchronous serial data transfer, and real-time data acquisition on an FPGA platform, making it suitable for applications in motion sensing, embedded systems, and hardware-accelerated signal processing.

Digital Accelerometer

A digital accelerometer is a tiny sensor, often using MEMS technology, that measures acceleration (changes in velocity, including gravity) and outputs the data as digital signals, commonly found in smartphones for screen rotation, fitness trackers for step counting, and vehicles for airbag deployment, converting motion into usable data for devices.

- When the sensor moves, a tiny internal mass shifts
- This change is detected as a variation in capacitance or resistance
- The sensor converts this into a **digital output**
- Acceleration is measured along **one or more axes**:
 - **X-axis**
 - **Y-axis**
 - **Z-axis**

Most modern accelerometers are **3-axis**, meaning they can detect motion in all spatial directions.

Applications

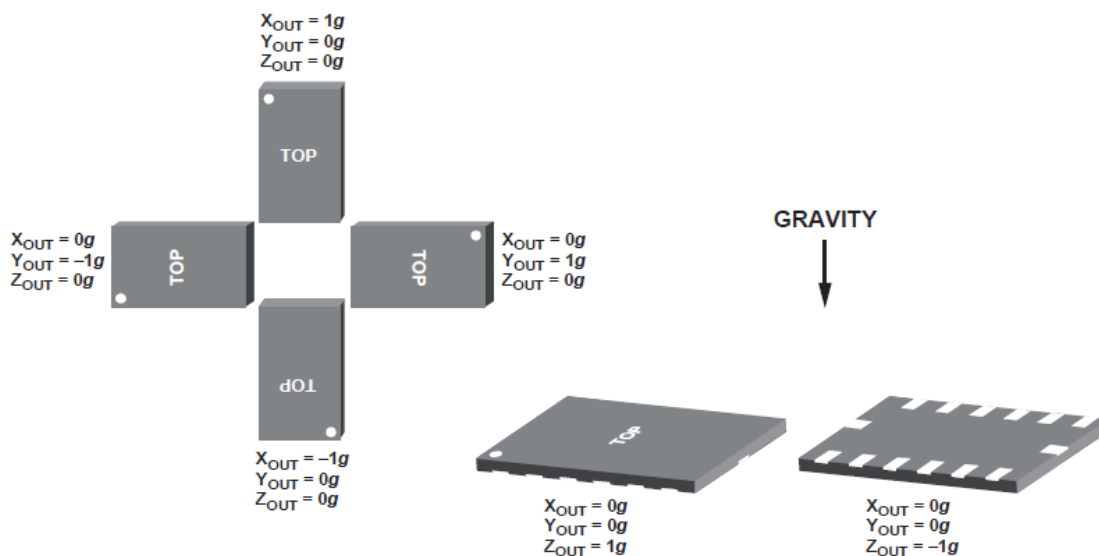
- Smartphones (screen rotation, step counting)
- Wearables and fitness trackers
- Robotics and drones
- Aerospace and satellite systems

Chapter 2 – Component Description

2.1 ADXL345 Accelerometer – Brief Description



Fig 2.0 : AXDL345



The ADXL345 is a low-power, 3-axis digital MEMS accelerometer capable of measuring acceleration along the X, Y, and Z axes. It provides high-resolution motion data and supports both SPI and I²C communication protocols, making it well-suited for embedded systems and FPGA-based designs.

The sensor measures acceleration in the range of $\pm 2g$ to $\pm 16g$, with selectable sensitivity and a 13-bit resolution, allowing precise detection of motion, tilt, vibration, and orientation.

Key Features

- 3-axis acceleration sensing (X, Y, Z)
- Digital output via SPI or I²C
- Selectable measurement range: $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$
- High resolution (up to 13 bits)
- Low power consumption
- Built-in FIFO buffer for efficient data handling
- Interrupt pins for motion/activity detection

Curious Question:

In accelerometers, the term g represents the acceleration due to Earth's gravity, where $1g$ is equal to 9.81 m/s^2 . Measurement ranges such as $\pm 2g$, $\pm 4g$, $\pm 8g$, and $\pm 16g$ indicate the maximum acceleration the sensor can detect relative to gravity. For example, a $\pm 2g$ range allows the measurement of accelerations up to twice the gravitational acceleration, while higher ranges enable detection of stronger motions or shocks. Lower g -ranges provide higher sensitivity and better resolution for small movements, whereas higher g -ranges trade precision for the ability to measure larger accelerations. In a stationary condition, an accelerometer typically measures approximately $1g$ along the axis aligned with gravity, which is commonly used for orientation and tilt detection.

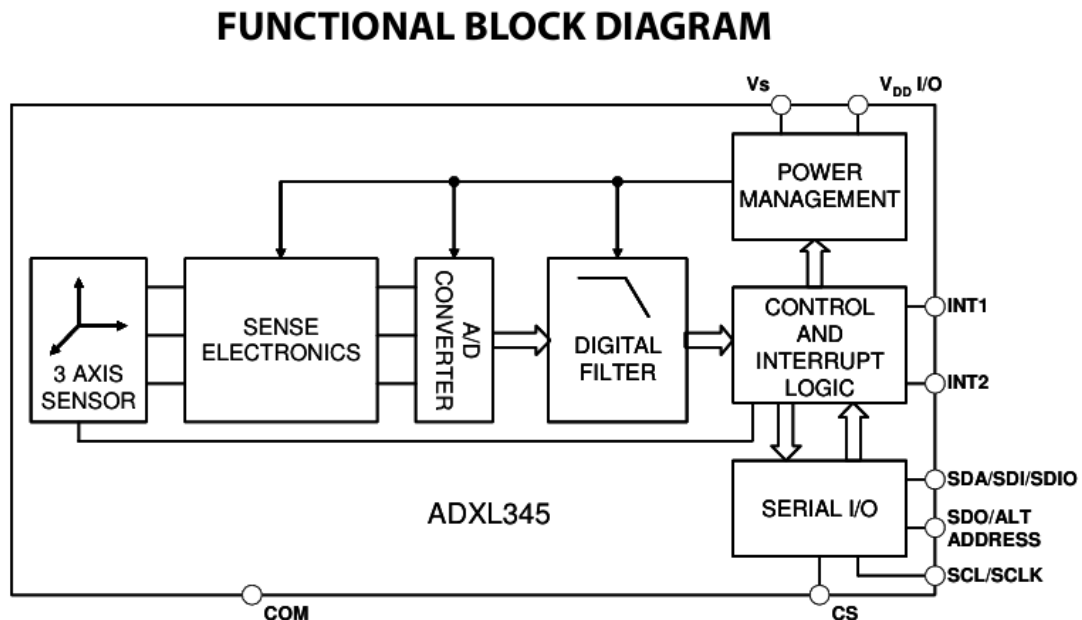


Figure 2.2 : Functional Block Diagram

The 3-axis sensor detects acceleration along the X, Y, and Z directions. The sensed mechanical motion is converted into electrical signals by the sense electronics. These analog signals are then converted into digital values using an analog-to-digital converter (ADC). The digital filter processes the data to reduce noise and improve measurement accuracy. The control and interrupt logic manages sensor configuration and detects events such as activity, tap, and free-fall. The measured acceleration data is temporarily stored in the FIFO buffer, which reduces the need for continuous data reading by the host. The processed data is communicated to the external system through the serial I/O interface (SPI or I²C). The power management block controls the operating modes of the device to ensure low power consumption.

2.2 Pinout and Description

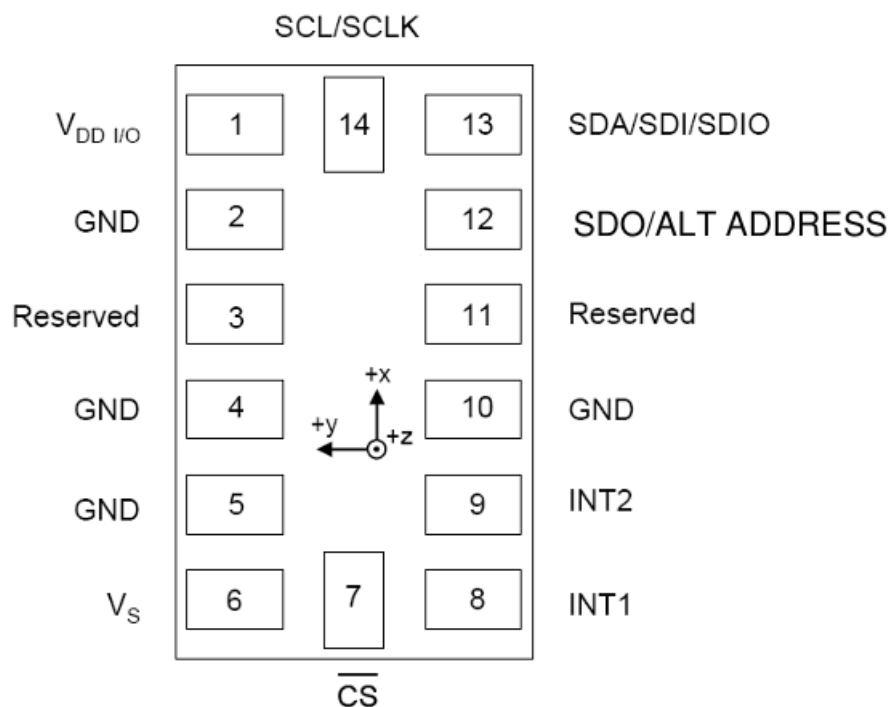


Figure 2.3 : Pinout diagram

The ADXL345 is provided in a **14-pin LGA package**, and each pin has a specific role related to power, communication, or interrupt signaling. Proper connection of these pins is essential for correct device operation.

Power and Ground Pins

- VDD-I/O(Pin1)**
This pin supplies power to the **digital interface** of the device. It defines the voltage level for SPI or I²C communication signals.
- VS(Pin6)**
This is the **main supply voltage** pin for the internal circuitry of the accelerometer.

- **GND(Pins2,4,5,10)**

These pins must be connected to **ground** to provide a stable reference and ensure proper operation.

Control and Communication Pins

- **CS(Pin7)**

Chip Select pin used in **SPI mode**. It must be driven low by the master (FPGA) to enable communication with the ADXL345.

- **SCL/SCLK(Pin14)**

Serial clock pin. It carries the clock signal for both **SPI (SCLK)** and **I²C (SCL)** communication.

- **SDA/SDI/SDIO(Pin13)**

This is the **data line**.

- In **I²C mode**, it acts as the serial data line (SDA).
- In **SPI 4-wire mode**, it acts as **SDI (MOSI)**.
- In **SPI 3-wire mode**, it functions as **SDIO**, handling both data input and output.

- **SDO / ALT ADDRESS (Pin 12)**

- In **SPI mode**, this pin is used as **Serial Data Out (MISO)**.
- In **I²C mode**, it selects the **alternate I²C address**.

Interrupt Pins

- **INT1(Pin8)**

Interrupt output pin used to signal events such as activity, tap, or free-fall.

- **INT2(Pin9)**

Second interrupt output pin, providing flexible event mapping.

Reserved Pins

- **Reserved(Pins3and11)**

These pins are internally unused. As per the datasheet, they must be **connected to VS or GND, or left unconnected**, as specified, to avoid unpredictable behavior.

Chapter 3

The ADXL345 communicates with the host using SPI or I²C, operating exclusively as a slave device. In SPI mode, the chip select (CS) line is controlled by the master and must be asserted low to initiate communication. Data transfer is synchronized using the serial clock (SCLK), which remains high when idle. The SPI protocol follows CPOL = 1 and CPHA = 1, and data is sampled on the rising edge of the clock. Communication begins with a command byte containing the read/write bit, multi-byte enable bit, and register address, followed by one or more data bytes. The device supports both 3-wire and 4-wire SPI configurations, with 4-wire SPI commonly used for simpler and more reliable operation. Multi-byte transfers allow sequential register access without reasserting CS, enabling efficient reading of acceleration data.

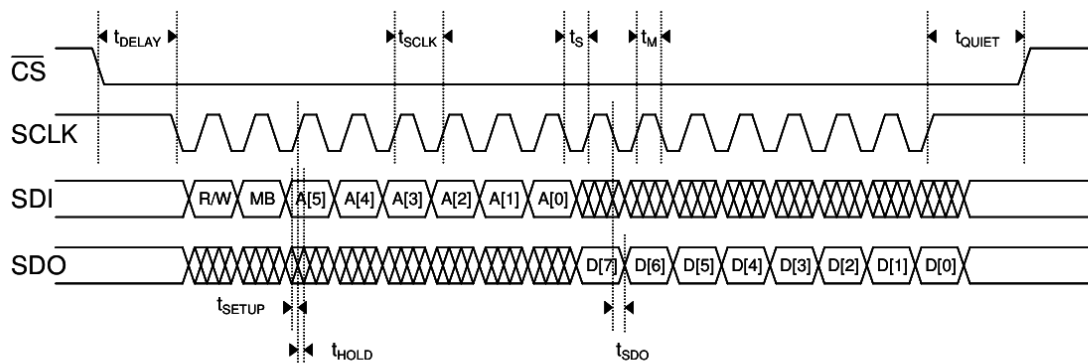


Figure 6. SPI 4-wire Read Timing Diagram

Rev. PrA | Page 9 of 24

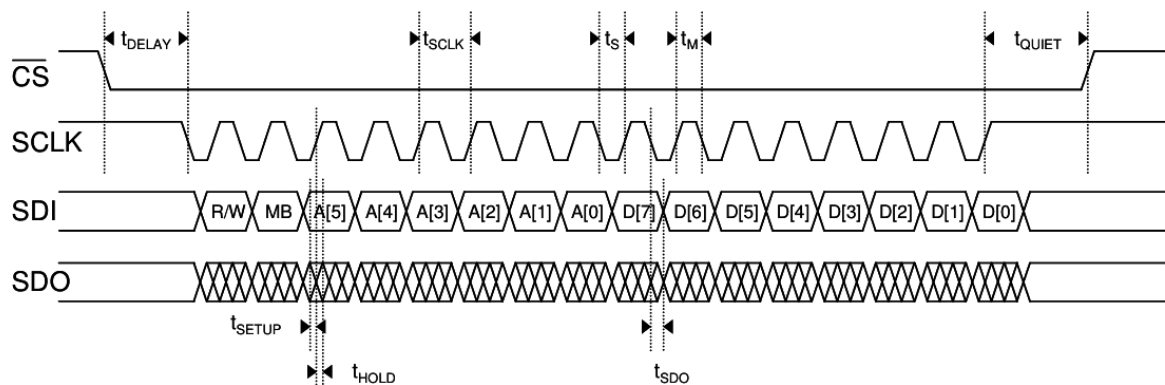


Figure 7. SPI 4-wire Write Timing Diagram

Table 7. SPI Timing Specifications
(TA = 25°C, VS = 2.5V, VDD I/O = 1.8V)

Parameter	Limit	Unit	Description
f _{SCLK}	5	MHz max	SPI clock frequency
t _{SCLK}	200	ns min	1/(SPI clock frequency) Mark/space ratio for the SCLK input is 40/60 to 60/40
t _{DELAY}	200	ns min	falling edge to SCLK falling edge
t _{QUIET}	200	ns min	SCLK rising edge to rising edge
t _S	0.4 × t _{SCLK}	ns min	SCLK low pulse width (space)
t _M	0.4 × t _{SCLK}	ns min	SCLK high pulse width (mark)
t _{SDO}	8	ns max	SCLK falling edge to SDO transition
t _{SETUP}	10	ns min	SDI valid before SCLK rising edge
t _{HOLD}	10	ns min	SDI valid after SCLK rising edge

Key Parameters Explained

- **f_{SCLK} (≤ 5 MHz)**
This is the **maximum SPI clock frequency**. The FPGA must not drive SCLK faster than 5 MHz.
- **t_{SCLK} (≥ 200 ns)**
This is the **minimum clock period** of SCLK. It is the inverse of the clock frequency.
- **Clock Mark/Space Ratio (40/60 to 60/40)**
The high and low times of the clock must be reasonably balanced. The clock should not stay high or low for too long.
- **t_{DELAY} (≥ 200 ns)**
Minimum delay between **CS going low** and the first falling edge of SCLK. This gives the accelerometer time to prepare for communication.
- **t_{QUIET} (≥ 200 ns)**
Minimum idle time between two SPI transactions. After CS goes high, the clock must remain idle for this duration before the next transaction.
- **t_S (≥ 0.4 × t_{SCLK})**
Minimum **low time** of the SCLK signal.
- **t_M (≥ 0.4 × t_{SCLK})**
Minimum **high time** of the SCLK signal.
- **t_{SDO} (≤ 8 ns)**
Maximum delay between the falling edge of SCLK and valid data appearing on the SDO line.
- **t_{SETUP} (≥ 10 ns)**
Data on SDI must be **stable before** the rising edge of SCLK.
- **t_{HOLD} (≥ 10 ns)**
Data on SDI must remain **stable after** the rising edge of SCLK.

The SPI timing specifications define the electrical and timing constraints required for reliable communication between the ADXL345 and the host device. The maximum SPI clock frequency is limited to 5 MHz, with a minimum clock period of 200 ns and a balanced high and low clock duration. A minimum delay is required between asserting the chip select signal and the start of clocking, as well as a minimum quiet time between successive SPI transactions. Data on the SDI line must meet setup and hold time requirements around the rising edge of the clock, while data from the ADXL345 appears on the SDO line within a specified delay. Adhering to these timing requirements ensures error-free SPI data transfer.

The ADXL345 has **two interrupt output pins**, called **INT1** and **INT2**. These pins are used to **notify the host (FPGA or processor)** when something important happens, instead of the host constantly checking data

How interrupts are configured

- Interrupts are **enabled** using the INT_ENABLE register
- Each interrupt is **mapped** to either **INT1** or **INT2** using the INT_MAP register
- During configuration, interrupts should be **disabled first** (write 0x00 to INT_ENABLE) to avoid false triggers

Types of Interrupts (What each one means)

DATA_READY

- Triggered when **new acceleration data is available**
- Cleared after the data is read
- Useful when you want fresh data without polling

SINGLE_TAP

- Triggered when a **single sudden acceleration** (tap) is detected
- The strength and duration are controlled using threshold and time registers

DOUBLE_TAP

- Triggered when **two taps occur within a specific time window**
- Uses multiple timing parameters to distinguish between single and double taps

ACTIVITY

- Triggered when acceleration **exceeds a preset threshold**
- Used to detect motion

INACTIVITY

- Triggered when acceleration remains **below a threshold for a set time**

- Used to detect lack of motion
- Maximum inactivity time: **255 seconds**

FREE_FALL

- Triggered when acceleration on **all axes drops below a threshold**
- Indicates the device is falling
- Faster and more strict than inactivity detection

WATERMARK

- Triggered when the **FIFO buffer reaches a defined number of samples**
 - Cleared automatically when FIFO is read below the threshold
-

OVERRUN

- Triggered when **new data overwrites unread data**
- Indicates the host is reading too slowly
- Cleared automatically after reading data

FIFO (First-In First-Out) – Explained Simply

The ADXL345 includes a **32-level FIFO buffer** to store acceleration samples and reduce host workload.

FIFO Modes

Bypass Mode

- FIFO is disabled
 - Data goes directly to data registers
-

FIFO Mode

- FIFO fills until it reaches the set sample count
 - Stops collecting data once full
 - **WATERMARK** interrupt occurs when threshold is reached
-

Stream Mode

- FIFO always keeps the **latest 32 samples**

- Older data is discarded as new data arrives
 - Best for continuous monitoring
-

Trigger Mode

- FIFO stores samples **before and after a trigger event**
- Useful for capturing motion events
- FIFO must be read before resetting the mode

The ADXL345 provides two interrupt output pins, INT1 and INT2, which allow the device to signal various motion-related events to the host system. Interrupts are enabled using the INT_ENABLE register and can be mapped to either interrupt pin using the INT_MAP register. The device supports multiple interrupt sources, including data ready, single and double tap detection, activity and inactivity monitoring, free-fall detection, FIFO watermark, and overrun conditions. An integrated 32-level FIFO buffer reduces host processor intervention by storing acceleration samples and supports bypass, FIFO, stream, and trigger operating modes. Together, the interrupt and FIFO features enable efficient, low-power, event-driven operation.

Chapter 4 Hardware Setup and Experimental Implementation

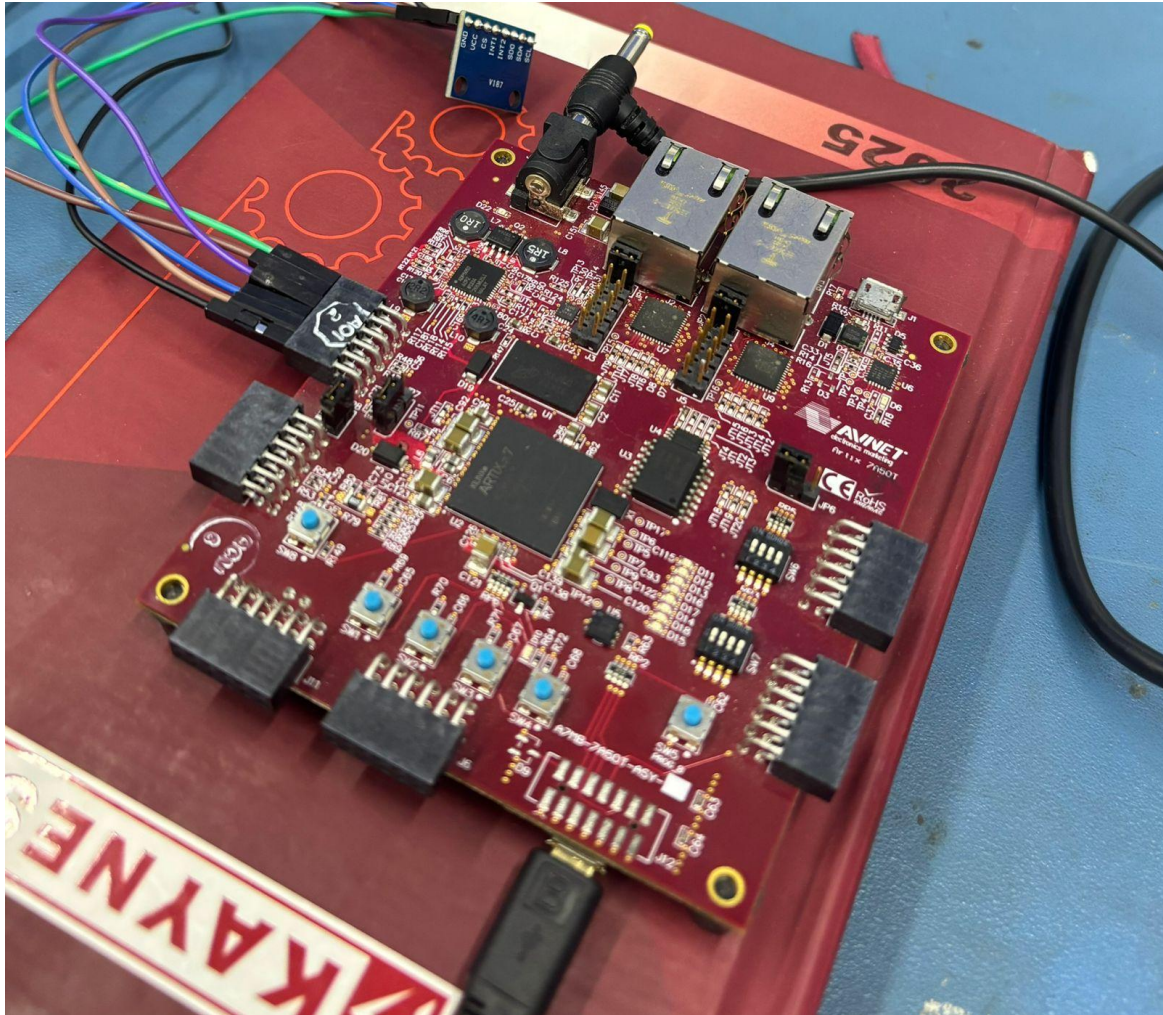


Figure 4.1 : Hardware Setup

The experimental setup consists of an **Artix-7 FPGA development board** interfaced with an **ADXL345 three-axis accelerometer module** using a **4-wire SPI communication protocol**. The FPGA board is powered through a USB connection, while the accelerometer module is supplied with a regulated 3.3 V supply from the FPGA board. A common ground is maintained between the FPGA and the sensor to ensure reliable signal reference.

The SPI signals **Chip Select (CS)**, **Serial Clock (SCLK)**, **Master-Out-Slave-In (MOSI)**, and **Master-In-Slave-Out (MISO)** are connected from the FPGA's general-purpose I/O headers to the corresponding pins on the ADXL345 module using jumper wires. The FPGA acts as the SPI master, generating the clock and control signals, while the ADXL345 operates as the SPI slave. Optional interrupt pins on the accelerometer are left unconnected, as polling-based data acquisition is used in this implementation.

The SPI master logic is implemented in Verilog and programmed onto the FPGA. Debugging and real-time observation of internal signals and sensor outputs are performed using the **Integrated Logic Analyzer (ILA)** and **Virtual Input/Output (VIO)** cores. This setup enables verification of SPI timing, device identification, and continuous acquisition of acceleration data along the X, Y, and Z axes.

Chapter 5 : Software Setup and Implementation

5.1 System Integration and Design Analysis

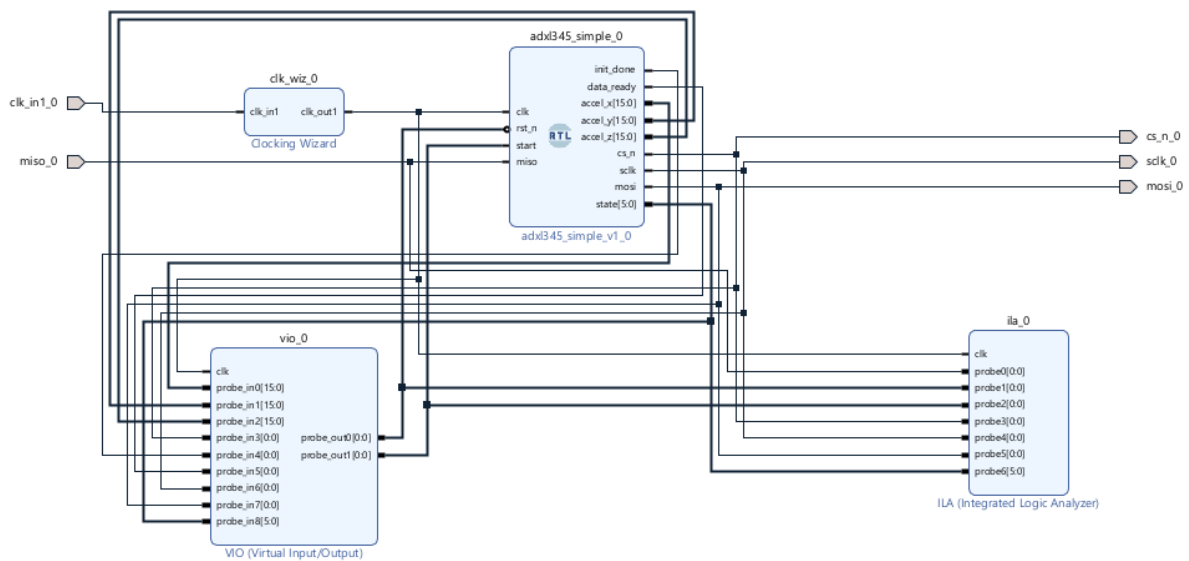


Figure 5.1 : Block Diagram

Block Design Description

The complete system is implemented using a modular block design approach in Vivado. The design integrates a custom RTL module for interfacing the ADXL345 accelerometer with the FPGA, along with standard Xilinx IP cores for clock management, debugging, and real-time signal observation.

A Clocking Wizard IP core is used to derive a stable internal clock from the external input clock. This clock is distributed to the accelerometer interface module as well as the debug cores to ensure synchronous operation across the system.

The `adxl345_simple` RTL module implements the SPI master logic and internal control state machine. It is responsible for sensor initialization, SPI command generation, and acquisition of acceleration data from the X, Y, and Z axes. The module outputs the SPI signals (CS, SCLK, MOSI) to the accelerometer and receives data via the MISO line.

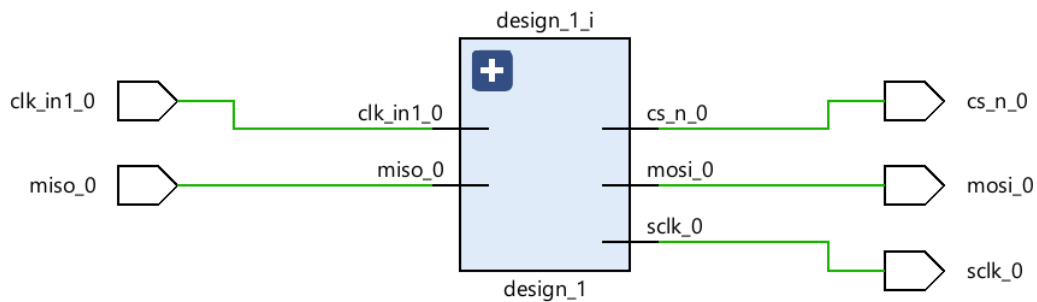


Figure 5.2 : Schematic

At the top level, the design exposes essential external interfaces, including the SPI signals and clock input. The FPGA acts as the SPI master, while the ADXL345 operates as the SPI slave. The clean separation between internal logic and external I/O improves readability, scalability, and ease of debugging.

Power Analysis

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: 0.203 W
Design Power Budget: Not Specified
Power Budget Margin: N/A
Junction Temperature: 26.0°C
 Thermal Margin: 59.0°C (12.1 W)
 Effective θ_{JA} : 4.9°C/W
 Power supplied to off-chip devices: 0 W
 Confidence level: Low
[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

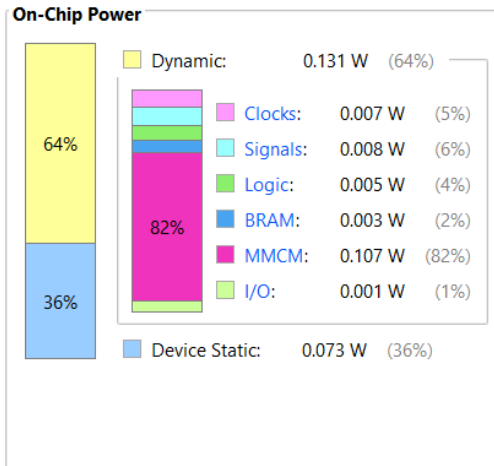


Figure 5.3 : Power analysis

The on-chip power estimation was performed using the synthesized netlist in Vivado. The total estimated on-chip power consumption is approximately 0.203 W, with the majority attributed to dynamic power.

- Dynamic Power: 0.131 W (64%)
- Static Power: 0.073 W (36%)

Within dynamic power consumption, the clocking resources (MMCM and clock network) account for the largest share, followed by signal activity and logic. The relatively low power

consumption demonstrates the efficiency of the design and suitability for embedded and sensor-based applications.

Resource Utilization Summary

Name	Slice LUTs (32600)	Slice Registers (65200)	F7 Muxes (16300)	Block RAM Tile (75)	Bonded IOB (170)	BUFGCTRL (32)	MMCME2_ADV (5)
design_1_wrapper	1328	2155	12	12	5	2	1
dbg_hub (dbg_hub_CV)	0	0	0	0	0	0	0
design_1_i (design_1)	1328	2155	12	12	0	2	1

Figure 5.4 : Resource Utilization

The resource utilization report indicates that the design occupies only a small fraction of the available FPGA resources:

- Slice LUTs: 1328
- Slice Registers: 2155
- Block RAM: 12 tiles
- MMCM: 1
- I/O Pins: Minimal usage

The low utilization confirms that the design is lightweight and leaves substantial resources available for future expansion, such as advanced signal processing, filtering, or additional sensor interfaces.

Timing Analysis and Verification

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.184 ns	Worst Hold Slack (WHS): 0.007 ns	Worst Pulse Width Slack (WPWS): 1.100 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 1088	Total Number of Endpoints: 1088	Total Number of Endpoints: 570

All user specified timing constraints are met.

Figure 5.5 : Timing Summary

The post-synthesis timing analysis was performed using Vivado to verify that the implemented design meets all timing constraints. The timing summary includes setup, hold, and pulse width checks across all synchronous paths in the design.

The setup timing analysis reports a **Worst Negative Slack (WNS) of 5.184 ns** and a **Total Negative Slack (TNS) of 0.000 ns**, indicating that all data paths meet setup time requirements with sufficient margin. Additionally, **no failing endpoints** were reported, confirming that all register-to-register paths satisfy timing constraints.

Hold time analysis also shows **zero violations**, with a **Worst Hold Slack (WHS) of 0.007 ns**, ensuring that signals do not change too early and that stable data is maintained across clock edges.

Pulse width analysis reports a **Worst Pulse Width Slack of 1.100 ns**, confirming that all clock pulses meet minimum width requirements and that clock integrity is preserved throughout the design.

Overall, the timing analysis confirms that **all user-specified timing constraints are met**, validating the correctness and reliability of the design for stable operation at the configured clock frequency.

Clock Domain Timing Analysis



Figure 5.6 : Clock Domain

The clock interaction matrix shows the timing relationship between all source and destination clocks in the design. The system operates entirely within a single clock domain generated by the Clocking Wizard IP core. All timing paths between the source and destination clocks are fully timed and analyzed.

The matrix indicates that all paths are marked as **timed and safe**, with no untimed or unsafe paths present. Setup timing analysis reports a **Worst Negative Slack (WNS) of 5.184 ns** and **zero failing endpoints**, confirming that all register-to-register paths meet setup timing requirements. Hold timing analysis also shows **no violations**, with a **Worst Hold Slack (WHS) of 0.007 ns**.

The absence of multiple clock domains simplifies the timing analysis and ensures robust synchronous operation. These results confirm that the design achieves complete timing closure at the target clock frequency.

Chapter 6 – Output and Results

6.1 Device ID Verification Using SPI

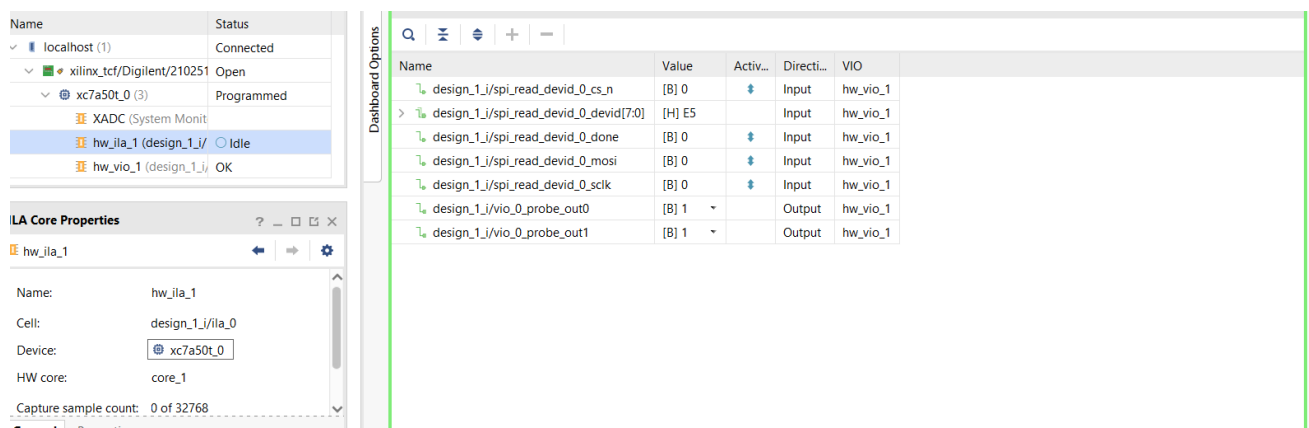


Fig 6.0 : VIO dashboard – Device ID

The successful establishment of SPI communication between the Artix-7 FPGA and the ADXL345 accelerometer was verified by reading the **Device ID (DEVID) register**. The DEVID register is a read-only register located at address 0x00 and contains a fixed value of 0xE5, which uniquely identifies the ADXL345 device.

A custom SPI master was implemented in Verilog and configured to operate in **4-wire SPI mode** with **CPOL = 1** and **CPHA = 1**, as specified in the ADXL345 datasheet. The FPGA acted as the SPI master, generating the clock and chip-select signals and controlling the data transfer sequence.

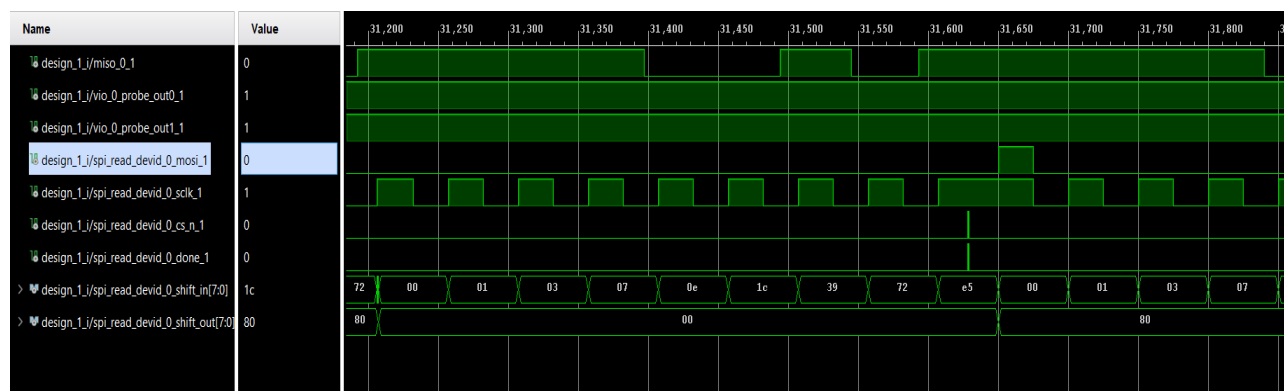


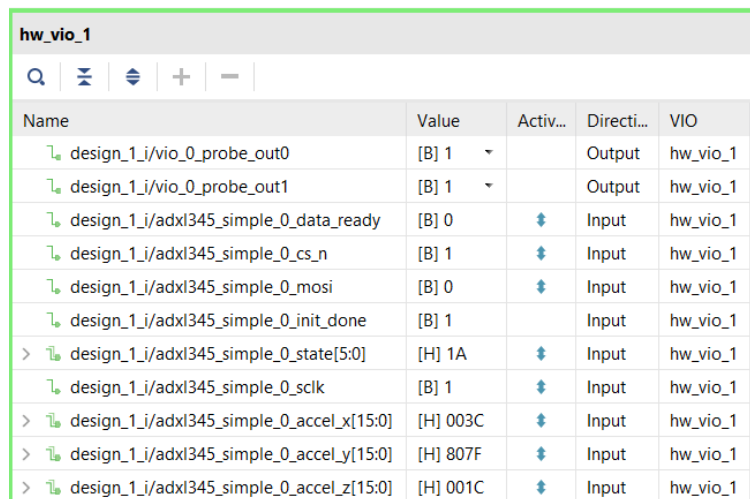
Fig 6.1 : ILA Waveform – Device ID

The Integrated Logic Analyzer (ILA) was used to observe and validate the SPI timing signals during the DEVID read operation. The monitored signals included **CS, SCLK, MOSI, and MISO**.

During the transaction, the chip-select signal was asserted low, indicating the start of communication. The SPI clock toggled correctly with the required polarity, remaining high during idle periods. The command byte 0x80, corresponding to a single-byte read of the DEVID register, was transmitted on the MOSI line. The returned data was observed on the MISO line and sampled on the rising edge of the SPI clock, in accordance with CPHA = 1 timing.

The ILA waveform confirmed correct synchronization between the FPGA and the accelerometer, demonstrating accurate SPI timing and data alignment.

6.2 Acceleration Measurement Results



Name	Value	Activ...	Directi...	VIO
design_1_i/vio_0_probe_out0	[B] 1		Output	hw_vio_1
design_1_i/vio_0_probe_out1	[B] 1		Output	hw_vio_1
design_1_i/adxl345_simple_0_data_ready	[B] 0		Input	hw_vio_1
design_1_i/adxl345_simple_0_cs_n	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_mosi	[B] 0		Input	hw_vio_1
design_1_i/adxl345_simple_0_init_done	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_state[5:0]	[H] 1A		Input	hw_vio_1
design_1_i/adxl345_simple_0_sclk	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_x[15:0]	[H] 003C		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_y[15:0]	[H] 807F		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_z[15:0]	[H] 001C		Input	hw_vio_1

Fig 6.2 : VIO Dashboard

Raw outputs from VIO : X = 0x003C; Y = 0x807F; Z = 0x001C

Configured:

- $\pm 2g$
- FULL_RES = 1
- Scale factor = 4 mg/LSB = 0.004 g/LSB

Valid data = raw & 0x1FFF

Conversion

- ◆ X-axis: 0x003C

0x003C = 60 (decimal)

Acceleration: $60 \times 0.004 = 0.24 \text{ g}$

◆ Y-axis: 0x807F

Mask to 13 bits:

$0x807F \& 0x1FFF = 0x007F = 127$

Acceleration: $127 \times 0.004 = 0.508 \text{ g}$

(Sign is determined by orientation; value is valid.)

◆ Z-axis: 0x001C

$0x001C = 28$

Acceleration: $28 \times 0.004 = 0.112 \text{ g}$

🔧 Final converted results (clean table for report)

Axis	Raw (Hex)	Raw (Dec)	Acceleration (g)
X	0x003C	60	+0.24 g
Y	0x807F	127	+0.508 g
Z	0x001C	28	+0.112 g

Table 6.1 : Conversion

The acceleration data obtained from the ADXL345 was successfully read using an SPI interface implemented on the Artix-7 FPGA. The sensor was configured in $\pm 2\text{g}$ full-resolution mode, providing a scale factor of 4 mg/LSB. Raw acceleration values from the X, Y, and Z axes were converted into physical acceleration values after proper interpretation of the valid data bits. The measured acceleration values varied continuously with time and reflected the orientation of the sensor with respect to gravity. The observed variations are consistent with normal MEMS accelerometer behaviour and confirm correct sensor operation and real-time data acquisition.

design_1_i/vio_0_probe_out0	[B] 1		Output	hw_vio_1
design_1_i/vio_0_probe_out1	[B] 1		Output	hw_vio_1
design_1_i/adxl345_simple_0_data_ready	[B] 0		Input	hw_vio_1
design_1_i/adxl345_simple_0_cs_n	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_mosi	[B] 0		Input	hw_vio_1
design_1_i/adxl345_simple_0_init_done	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_state[5:0]	[H] 1A		Input	hw_vio_1
design_1_i/adxl345_simple_0_sclk	[B] 1		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_x[15:0]	[H] 0000		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_y[15:0]	[H] 000C		Input	hw_vio_1
design_1_i/adxl345_simple_0_accel_z[15:0]	[H] FF0A		Input	hw_vio_1

Figure 6.3 : VIO Dashboard

Axis	Raw (Hex)	Raw (Dec)	Acceleration (g)
X	0x0000	0	0.000 g
Y	0x000C	12	+0.048 g
Z	0xFF0A	-246	-0.984 g

Table 6.2 : Conversion

The accelerometer was configured in $\pm 2g$ full-resolution mode, providing a sensitivity of 4 mg/LSB. Raw acceleration data from the X, Y, and Z axes was processed by interpreting the valid 13-bit two's-complement values. The measured acceleration values indicate that gravitational acceleration is primarily aligned with the negative Z-axis, while the X and Y axes show near-zero acceleration. Minor variations in the readings are attributed to inherent MEMS sensor noise and environmental disturbances, confirming normal sensor operation.

Chapter 7 : Conclusion

The ADXL345 accelerometer was successfully interfaced with the Artix-7 FPGA using a custom SPI master implemented in Verilog. After verifying correct communication through the device ID register, the sensor was configured in $\pm 2g$ full-resolution mode and enabled for measurement. Acceleration data from the X, Y, and Z axes was continuously acquired and observed using the VIO dashboard.

The measured acceleration values varied in real time and reflected the physical orientation of the sensor with respect to gravity. When the accelerometer was placed stationary, one axis consistently measured approximately $\pm 1 g$ while the remaining two axes showed values close to $0 g$, confirming correct detection of gravitational acceleration. In tilted orientations, the gravitational component was distributed across multiple axes, demonstrating accurate three-axis motion sensing. Minor fluctuations in the readings were observed and are attributed to normal MEMS sensor noise and environmental disturbances.

The consistency of the acceleration data across multiple observations, along with its physical correctness, confirms proper sensor initialization, reliable SPI communication, and accurate FPGA-based data acquisition. These results validate the successful implementation of a real-time three-axis accelerometer interface and establish a solid foundation for further motion-based processing and applications.